

## Register Machine (machine)

Bei Renovierungsarbeiten an den Bock-Kasematten – einem Netz unterirdischer Befestigungsanlagen in Luxemburg-Stadt – haben Arbeiter eine uralte Schatzkiste ausgegraben, die durch einen recht eigenartigen Mechanismus versiegelt war.

Die Inschrift auf dem Deckel bezeichnet sie als *Registermaschine* und erklärt, dass sie über 64 Register verfügt, die von 0 bis 63 nummeriert sind und von denen jedes eine nicht-negative ganze Zahl enthalten kann. Anfangs enthalten alle Register den Wert 0.

Über eine Reihe von Drehreglern und Tasten kann man ein *Programm* eingeben, das aus einer Folge von Befehlen besteht. Der erste eingegebene Befehl wird mit 0 bezeichnet, der nächste mit 1 und so weiter. Es gibt drei Arten von Befehlen:

- **INCREMENT  $x, i$**  — Der im Register  $x$  gespeicherte Wert wird um 1 erhöht, anschliessend verarbeitet die Maschine die mit  $i$  gekennzeichnete Anweisung.
- **DECREMENT  $x, i, j$**  — Ist der im Register  $x$  gespeicherte Wert 0, verarbeitet die Maschine die mit  $i$  gekennzeichnete Anweisung. Andernfalls wird der Wert im Register  $x$  um 1 verringert, anschliessend führt die Maschine den mit  $j$  gekennzeichneten Befehl aus.
- **HALT** — Die Maschine beendet die Ausführung der Befehle.

Die Maschine beginnt mit der Ausführung ab dem mit 0 gekennzeichneten Befehl. Sobald sie anhält, öffnet sie die Schatzkiste, wenn der im Register 0 gespeicherte Wert genau  $N$  beträgt.

Wenn sie nach der Ausführung von 50 000 000 Befehlen nicht anhält oder wenn sie versucht, einen Befehl auszuführen, der nicht existiert, wird die Maschine stattdessen katastrophal versagen und die Schatzkiste für immer versiegeln.

Ihre Aufgabe ist es, ein Programm einzugeben, das die Schatzkiste öffnet. Da die Maschine empfindlich ist, sollte das Programm so kurz wie möglich sein, und auf keinen Fall jedoch mehr als 100 Befehle umfassen.

## Implementierung

Du musst eine einzelne `.cpp`-Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `folding.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

C++

```
void program_machine(int N)
```

- Ganzzahl  $N$  repräsentiert die Zahl, welche in Register 0 gespeichert werden soll, wenn die Maschine beendet wird.

Dein Programm kann die folgenden Funktionen höchstens 100 Mal aufrufen. Diese Funktionen sind im Grader bereits definiert.

C++

```
void add_increment(int x, int i)
```

|     |                                                      |
|-----|------------------------------------------------------|
| C++ | <code>void add_decrement(int x, int i, int j)</code> |
| C++ | <code>void add_halt()</code>                         |

## Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der Beispielgrader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `program_machine` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus eine Zeile, die die Ganzzahl  $N$  enthält.

Die Ausgabe besteht aus einer Zeile:

- Falls dein Programm die Schatzkiste erfolgreich öffnet, so ist die Ausgabe `OK L`, wobei  $L$  die Länge deine Programms ist.
- Ansonsten beginnt die Ausgabe mit `WA`, gefolgt von einer kurzen Begründung (e.g. das Programm ist länger als 100 Befehle, das Programm wurde nicht rechtzeitig beendet, etc.)

## Einschränkungen

- $1 \leq N \leq 1\,000\,000$ .

## Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen.

In Teilaufgabe 3 wird die Punktzahl anhand der Länge deines Programms berechnet. Sei  $L$  die maximale Länge deines Programms über alle Testfälle dieser Teilaufgabe. Die Punktzahl für diese Teilaufgabe ist  $\min(77, 100 - L)$ . Insbesondere gilt: Um die ganzen 77 Punkte für diese Teilaufgabe zu erhalten, darf dein Programm höchstens 23 Befehle auf jedem Testfall dieser Teilaufgabe verwenden.

- **Teilaufgabe 0 [ 0 Punkte]:** Beispiel-Testfälle.
- **Teilaufgabe 1 [ 7 Punkte]:**  $N \leq 99$ .
- **Teilaufgabe 2 [16 Punkte]:**  $N$  ist eine Zweierpotenz (i.e.  $N = 2^k$  für eine Ganzzahl  $k \geq 0$ ).
- **Teilaufgabe 3 [77 Punkte]:** Keine weiteren Einschränkungen. (Beachte, dass die Bewertung für diese Teilaufgabe von der Länge deines Programms abhängt.)

## Beispiele

| Grader                          | Lösung                                                                                                     |
|---------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>program_machine(3)</code> | <pre> add_increment(1, 1) add_increment(0, 2) add_decrement(1, 4, 1) add_halt() add_increment(0, 3) </pre> |

## Grader

## Lösung

program\_machine(5)

```
add_increment(0, 1)
add_increment(0, 2)
add_increment(0, 3)
add_increment(0, 4)
add_increment(0, 5)
add_halt()
```

## Erklärung

Im ersten Beispiel,  $N = 3$ . Das Beispiel erstellt ein Programm mit 5 Befehlen:

|    |                   |
|----|-------------------|
| 0. | INCREMENT 1, 1    |
| 1. | INCREMENT 0, 2    |
| 2. | DECREMENT 1, 4, 1 |
| 3. | HALT              |
| 4. | INCREMENT 0, 3    |

Die folgende Tabelle zeigt die Bezeichnung jedes Befehls, den die Maschine verarbeitet, sowie den Inhalt der Register 0 und 1 danach:

| Befehl | Register 0 | Register 1 |
|--------|------------|------------|
| —      | 0          | 0          |
| 0      | 0          | 1          |
| 1      | 1          | 1          |
| 2      | 1          | 0          |
| 1      | 2          | 0          |
| 2      | 2          | 0          |
| 4      | 3          | 0          |
| 3      | 3          | 0          |

Im zweiten Beispiel,  $N = 5$  und das gezeigte Beispielprogramm verwendet 6 Befehle.